1 → 2 → 4 → null
→ 1 → 4

Page No.
Date

Prims

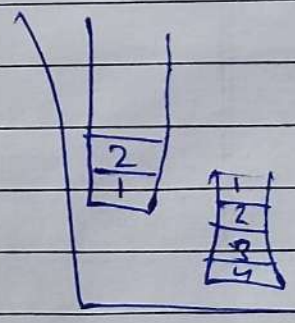
algo

→ We will convert every edge into adjacency list

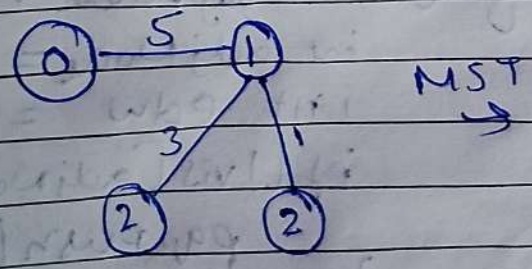
→ Initialize visited array, & create a min-heap (weight, node) and then start from node 0;

→ While heap is not empty
- extract minimum heap
- if node already visited, skip
- mark visited & add weight of that node
- push all adjacent edge of that node

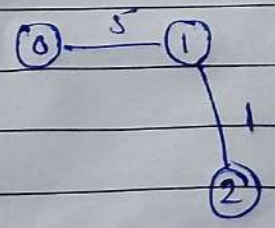
continue for all node



→ ~~Vector~~ ~~Prims~~ For prim the
TC → $O(V + E \log V)$
SC → $O(V)$



MST →



total wt = 6

Date

1 →

Prims PseudoCode.

→ Create an adjacency list from the given set of nodes

→ priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;

vector<int> vis(n, 0);

pq.push({0, 0});

long long sum = 0;

while(!pq.empty()) {

int wt = pq.top().first();

int node = pq.top().second();

pq.pop();

if (vis[node]) continue;

~~for(auto adjNode : adj[node]){~~

~~int adjWt =~~

vis[node] = 1;

sum += wt;

for(auto ~~adjNode~~ it : adj[node]){

int adjNode = it[0];

int edw = it[1];

if(!vis[adjNode]){

pq.push({edw, adjNode});

}

}

}, return sum

Teacher's Signature.....

23BCS11262
Gautam Prajapat
KRG-2A

Page No.
Date

Q. TSP

Algorithm

→ We Initialize DP table with INT_MAX
→ Starting from city 0 → dp[1][0] = 0
→ ie for a given State we have
that We will try all the given cities.

for every mask
for every current city we have
if is in mask
→ Try all next cities not in mask
→ update

Add cost to return to city 0.
Take minimum overall ending cities

PseudoCode:

```
int tsp(vector<vector<int>> & cost){
    int n = cost.size();
    int N = 1<<n
    vector<vector<int>> dp(N, vector<int>(n,-1))
    dp[1][0] = 0
```

```
for(int m=1 ; m<N ; m++){
    for ( u=0 → n){
        if(m & (1<<u)){
            for (v=0 → n){
                if(!mask & (1<<v))
                    int nmask = mask | (1<<v);
                    dp[nmask][v] = min(dp[nmask]
    [v], dp[mask][u] + cost(u)[v]);

    }.

    ans = 1e9
    for (u=0 → n){
        ans = min(ans, dp[N-1][u] + cost
                [u][0]);
    }
    return ans;
```

$T(c) = O(n^2 . 2^n)$

$S(c) = O(n . 2^n)$